

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Малышенко А. Р.

Группа К34392

Проверил:

Добряков Д. И.

Санкт-Петербург

2024 г.

Содержание отчета

1) Постановка задачи	3
2) Ход работы	3
3) Вывод	11

1) Постановка задачи

В рамках данной лабораторной работы Вам предложено выбрать один из нескольких вариантов. Выбранный вариант останется единственным на весь курс и будет использоваться в последующих лабораторных работах.

По выбранному варианту необходимо будет реализовать RESTful API средствами express + typescript (используя ранее написанный boilerplate)

Был выбран **вариант №3**:

Сервис-помощник в планировании путешествий. Требуемый функционал: регистрация, авторизация, создание профиля, создание поездки с заполнением информации о ней, поиск по существующим предложениям для поездок, предложения для новой созданной поездки на основе интересов пользователя.

2) Ход работы

Всего было создано 13 моделей (рисунок 1) и в качестве примера приведена модель поездок (рисунок 2).

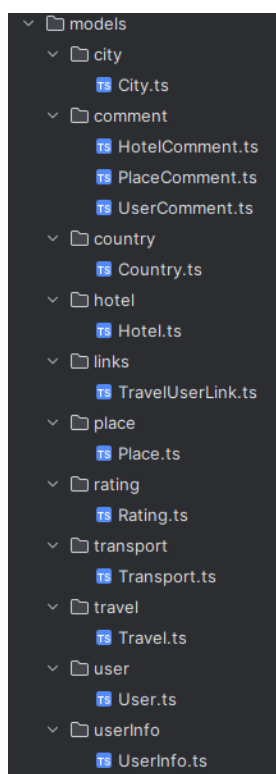


Рисунок 1 – 13 моделей

```

export type TravelAttributes = {
  id: number,
  user_id: number,
  transport_id: number,
  departure_city: number,
  destination_city: number,
  departure_date: string,
  expected_arrival_date: string,
  actual_arrival_date: string,
  state: string,
  seats: string,
}

export type TravelCreate = Optional<TravelAttributes, "id">

@Table Show usages
export class Travel extends Model<TravelAttributes, TravelCreate> {
  @ForeignKey(() => Transport)
  @Column
  transport_id!: number

  @ForeignKey(() => User)
  @Column
  user_id!: number

  @ForeignKey(() => City)
  @Column
  departure_city!: number

  @ForeignKey(() => City)
  @Column
  destination_city!: number

  @AllowNull(false)
  @Column
  departure_date!: number

  @AllowNull(false)
  @Column
  expected_arrival_date!: number

  @Column
  actual_arrival_date!: number

  @Column
  state!: string

  @Column
  seats!: string

  @BelongsToMany(() => User, () => TravelUserLink)
  users!: User[]
}

export default Travel Show usages

```

Рисунок 2 – модель путешествий (travel)

Далее для каждой модели необходимо создать основной роутер в /routes/index.ts (рисунок 3), а также роутеры для каждой модели отдельно, в качестве примера используется роутеры модели travel (рисунок 4). Стоит заметить, что у некоторых роутеров в примере есть часть кода «validateJWT» это значит, что для получения ответа по этому запросу, необходимо отправлять jwt токен, то есть быть авторизованным.

```
const router: express.Router = express.Router()

router.use('/cities', cityRoutes)
router.use('/comments/hotel', hotelCommentRoutes)
router.use('/comments/place', placeCommentRoutes)
router.use('/comments/user', userCommentRoutes)
router.use('/countries', countryRoutes)
router.use('/hotels', hotelRoutes)
router.use('/links/travel_user', travelUserLinkRoutes)
router.use('/places', placeRoutes)
router.use('/transports', transportRoutes)
router.use('/travels', travelRoutes)
router.use('/users', userRoutes)
router.use('/users/info', userInfoRoutes)

export default router
```

Рисунок 3 – роутеры в index.ts

```
const router = express.Router()
const controller = new TravelController(new TravelService())

router.route("/add").post(validateJWT, controller.postCreate)
router.route("/list").get(controller.listFindAll)
router.route("/list/by/dep_city/:dep_city_id").get(controller.listFindAllByDepartureCityId)
router.route("/list/by/dep_and_dest_city/:dep_city_id/:dest_city_id")
  .get(controller.listFindAllByDestinationAndDepartureCityId)
router.route("/one/:id").get(controller.getFindById)
router.route("/list/by/seats_number/:seats_number").get(controller.listFindByNumberOfSeats)
router.route("/delete/:id").delete(validateJWT, controller.deleteById)
router.route("/update/:id").patch(validateJWT, controller.patchUpdateById)

export default router
```

Рисунок 4 – роутеры модели travel

Теперь для роутеров прописываем контроллеры (рисунки 4, 5, 6, 7), сервисы (рисунки 8, 9) и маперы (рисунок 10) для каждой модели.

```

class TravelController { Show usages
  private readonly travelService: TravelService
  private readonly travelMapper = new TravelMapper()

  constructor(travelService: TravelService) { Show usages
    this.travelService = travelService
  }

  getFindById = async (request: Request, response: Response) => { Show usages
    const id = Number(request.params.id)
    const travel = await this.travelService.findById(id)
    if (!travel) {
      return response.status(404).send()
    }

    const dto = this.travelMapper.travelToDict(travel)
    return response.status(200).json(dto)
  }

  postCreate = async (request: Request, response: Response) => { Show usages
    const body: TravelCreate = request.body

    try {
      const createdTravel = await this.travelService.create(body)
      const dto = this.travelMapper.travelToDict(createdTravel)
      return response.status(201).send(dto)
    } catch (e: any) {
      return response.status(400).send({"message": e.message})
    }
  }

  patchUpdateById = async (request: Request, response: Response) => { Show usages
    const body: TravelCreate = request.body
    const id = Number(request.params.id)
    try {
      const updatedTravel = await this.travelService.updateById(id, body)
      return response.status(200).send({"message": "Updated"})
    } catch (e: any) {
      return response.status(400).send({"message": e.message})
    }
  }
}

```

Рисунок 4 – контроллер модели travel

```

listFindAll = async (request: Request, response: Response) => { Show usages
  const travellist = await this.travelService.findAll()
  if (!travellist) {
    return response.status(404).send()
  }

  let data: any[] = []

  travellist.forEach( travel => {
    data.push(this.travelMapper.travelToDict(travel))
  })

  return response.status(200).json(data)
}

listFindAllByDepartureCityId = async (request: Request, response: Response) => { Show usages
  const dep_city_id = Number(request.params.dep_city_id)
  const travellist = await this.travelService.findAllByDepartureCityId(dep_city_id)
  if (!travellist) {
    return response.status(404).send()
  }

  let data: any[] = []

  travellist.forEach( travel => {
    data.push(this.travelMapper.travelToDict(travel))
  })

  return response.status(200).json(data)
}

```

Рисунок 5 – контроллер модели travel

```

listFindAllByDestinationAndDepartureCityId = async (request: Request, response: Response) => { Show usages
  const dep_city_id = Number(request.params.dep_city_id)
  const dest_city_id = Number(request.params.dest_city_id)
  const travellist = await this.travelService.findAllByDestinationAndDepartureCityId(dep_city_id, dest_city_id)
  if (!travellist) {
    return response.status(404).send()
  }

  let data: any[] = []

  travellist.forEach( travel => {
    data.push(this.travelMapper.travelToDict(travel))
  })

  return response.status(200).json(data)
}

```

Рисунок 6 – контроллер модели travel

```

deleteById = async (request: Request, response: Response) => { Show usages
    const id = Number(request.params.id)
    const deletedRows = await this.travelService.deleteById(id)
    if (deletedRows === 0) {
        return response.status(404).send()
    }

    return response.status(204).send()
}

listFindByNumberOfSeats = async (request: Request, response: Response) => { Show usages
    const seatsNumber = Number(request.params.seats_number)
    const travelList = await this.travelService.findAllByNumberOfSeats(seatsNumber)
    if (!travelList) {
        return response.status(404).send()
    }

    let data: any[] = []

    travelList.forEach( travel => {
        data.push(this.travelMapper.travelToDict(travel))
    })

    return response.status(200).json(data)
}

```

Рисунок 7 – контроллер модели travel

```

class TravelService implements CrudTravelInterface<number, number> { Show usages
    private travelRepository: Repository<Travel> = sequelize.getRepository(Travel)

    create(data: TravelCreate): Promise<Travel> { Show usages
        try {
            return this.travelRepository.create(data)
        } catch (e: any) {
            return Promise.reject(e)
        }
    }

    deleteById(id: number): Promise<number> { Show usages
        return this.travelRepository.destroy({
            where: {
                id: id
            }
        })
    }

    async findById(id: number): Promise<Travel | null> { Show usages
        return this.travelRepository.findOne({
            where: {
                id: id
            }
        })
    }
}

```

Рисунок 8 – сервис модели travel

```

    async findAll(): Promise<Array<Travel> | null> { Show usages
        return this.travelRepository.findAll()
    }

    async findAllByDepartureCityId(dep_city_id: number): Promise<Array<Travel> | null> { Show usages
        return this.travelRepository.findAll({
            where: {
                departure_city: dep_city_id
            }
        })
    }

    async findAllByDestinationAndDepartureCityId(dep_city_id: number, dest_city_id: number): Promise<Array<Travel> | null>
        return this.travelRepository.findAll({
            where: {
                departure_city: dep_city_id,
                destination_city: dep_city_id
            }
        })
    }

    async updateById(id: number, data: TravelCreate): Promise<Travel> { Show usages
        const result = await this.travelRepository.update(data, {
            where: {
                id: id
            },
            returning: true
        })

        if (result[0] === 0) {
            return Promise.reject({message: "Not found"})
        }

        return Promise.resolve(result[1][0])
    }

    async findAllByNumberOfSeats(numberOfSeats: number): Promise<Array<Travel> | null> { Show usages
        return this.travelRepository.findAll({
            where: {
                seats: numberOfSeats
            }
        })
    }
}

export default TravelService Show usages

```

Рисунок 9 – сервис модели travel

```

class TravelMapper { Show usages
  travelToDict(entity: Travel) { Show usages
    return {
      id: entity.id,
      user_id: entity.user_id,
      transport_id: entity.transport_id,
      departure_city: entity.departure_city,
      destination_city: entity.destination_city,
      departure_date: entity.departure_date,
      expected_arrival_date: entity.expected_arrival_date,
      actual_arrival_date: entity.actual_arrival_date,
      state: entity.state,
      seats: entity.seats,
    };
  }
}

export default TravelMapper Show usages

```

Рисунок 10 – мапер модели travel

Также необходимо добавить функции, отвечающие за генерацию и проверку jwt токенов (рисунок 11).

```

const validateJWT = (req: Request, res: Response, next: NextFunction) => { Show usages
  const headerToken = req.headers.authorization
  if (headerToken != undefined && headerToken.startsWith("Bearer ")) {
    try {
      const bearerToken = headerToken.slice(7)
      const res = jwt.verify(bearerToken, process.env.JWTKEY || "error")
      next()
    } catch (e) {
      res.status(401).send({ "error": e })
    }
  } else {
    res.status(401).json({ "msg": "Access token denied" })
  }
}

export default validateJWT Show usages

```

Рисунок 11 – проверка jwt токена

Далее необходимо добавить подключение к базе данных и добавить в базу данных все созданные ранее модели (рисунок 12), все данные для подключения к бд берутся из .env файла.

```

const sequelize = new Sequelize({
  username: process.env.username,
  password: process.env.password,
  database: process.env.database,
  host: process.env.host,
  dialect: 'sqlite',
  storage: 'database.sqlite'
})

const models = [
  User,
  City,
  HotelComment,
  PlaceComment,
  UserComment,
  Country,
  Hotel,
  TravelUserLink,
  Place,
  Transport,
  Travel,
  UserInfo
]

sequelize.addModels(models)
sequelize.sync().then(() => console.log("models are synced")).catch((error) => console.log(error))

export default sequelize Show usages

```

Рисунок 12 – подключение к базе данных

3) Вывод

В ходе лабораторной работы №2 был реализован RESTful api с использованием express и typescript при помощи boilerplate реализованного в рамках лабораторной работы №1.